Additional material DM

Caroline König

11/11/2024

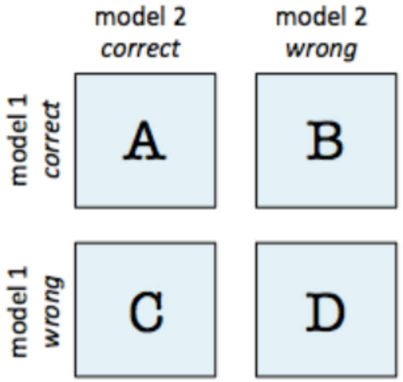
Outline

1. McNemar’s test
2. Example Naïve Bayes
3. Generative and discriminative classifiers

Testing for statistical difference between classifiers

1. McNemar’s Test

- Statistical test used to compare the predictive accuracy of two models.
- Null hypothesis: None of the two models performs better than the other.
- Significance threshold = 0.05



$$\chi^2 = \frac{(|\, b - c \,| - 1)^2}{(b + c)}.$$

Example - Which model is better?

- Model 1 accuracy= 99,6%
- Model 2 accuracy= 99,7%

A

	model 1 *correct*	model 1 *wrong*
model 2 *correct*	9959	11
model 2 *wrong*	1	29

B

	model 1 *correct*	model 1 *wrong*
model 2 *correct*	9945	25
model 2 *wrong*	15	15

Scenario B

	correct	wrong
model 2 correct	9945	25
model 2 wrong	15	15

$$\chi^2 = \frac{(|b - c| - 1)^2}{(b + c)}.$$

```
import numpy as np

tb_b = np.array([[9945, 25],
                 [15, 15]])
```

To test the null hypothesis that the predictive performance of two models are equal (using a significance level of $\alpha = 0.05$), we can conduct a corrected McNemar Test for computing the chi-squared and p-value as follows:

```
from mlxtend.evaluate import mcnemar

chi2, p = mcnemar(ary=tb_b, corrected=True)
print('chi-squared:', chi2)
print('p-value:', p)
```

Since the p-value is larger than our assumed significance threshold, we cannot reject the null hypothesis and assume there is no significant difference between the two predictive models.

Scenario A

The sample size is relatively small (b + c = 11 + 1 = 12), and smaller than the recommended 25 to approximate the computed chi-square value by the chi-square distribution well.
Compute the exact p-value from the binomial distribution:

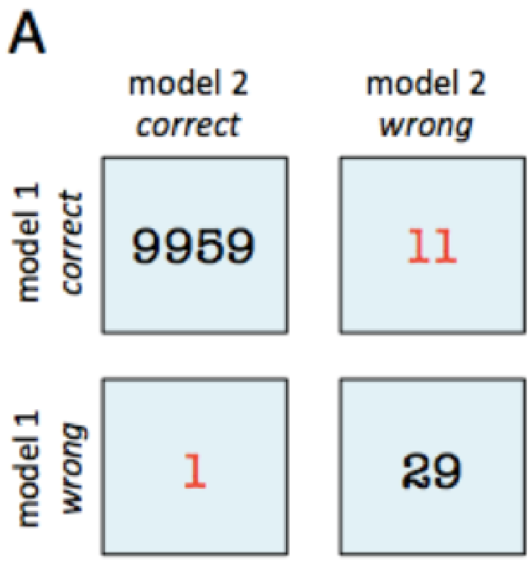
```
from mlxtend.evaluate import mcnemar
import numpy as np

tb_a = np.array([[9959, 11],
                 [1, 29]])

chi2, p = mcnemar(ary=tb_a, exact=True)

print('chi-squared:', chi2)
print('p-value:', p)
```

```
chi-squared: None
p-value: 0.005859375
```



P-value<α, reject the null hypothesis, assume there is a significant difference betw. the two predictive models.

Example Naïve Bayes

Naïve Bayes

Calculate the probability of the observation X for each class C.

$$\boxed{c_{NB} = \underset{c_j \in C}{\operatorname{argmax}} \frac{P(x_1, x_2, \ldots, x_n \mid c_j) P(c_j)}{P(x_1, x_2, \ldots, x_n)} = \underset{c_j \in C}{\arg\max}\, P(c_j) \prod_i P(x_i \mid c_j)}$$

Dataset

Color	Number Legs	Heigth	Class
Black	2	tall	M
White	2	small	M
black	4	small	M
Black	4	small	M
Black	4	tall	H
White	2	tall	H
Black	4	small	H
White	4	tall	H

P(M)= 0,5
P(H)= 0,5

P(Color=black|M) = 0.75
P(Color=white|M) = 0.25
P(Color=black|H) = 0.5
P(Color=white|H) = 0.5
P(Legs=2|M)= 05
P(Legs=4|M)= 0.5
P(Legs=2|H)= 0.25
P(Legs=4|H)= 0.75
P (Height=tall|M)= 0.25
P (Height=small|M)= 0.75
P (Height=tall|H)= 0.75
P (Height=small|H)= 0.25

Class prediction for new observation

P(M)= 0,5
P(H)= 0,5

P(Color=black|M) = 0.75
P(Color=white|M) = 0.25
P(Color=black|H) = 0.5
P(Color=white|H) = 0.5
P(Legs=2|M)= 0.5
P(Legs=4|M)= 0.5
P(Legs=2|H)= 0.25
P(Legs=4|H)= 0.75
P (Height=tall|M)= 0.25
P (Height=small|M)= 0.75
P (Height=tall|H)= 0.75
P (Height=small|H)= 0.25

New observation X= (Color=white, Legs=2, Height= tall)

P(M|X) = P(M) * P(Color=white|M) * P(Legs=2|M) * P (Height=tall|M) =
0,5 * 0,25 * 0.5 * 0.25 = 0.015625

P(H|X) = P(H) * P(Color=white|H) * P(Legs=2|H) * P (Height=tall|H) =
0,5 * 0,5 * 0.25 * 0.75 = 0.046875

The naïve bayes model predicts class H for observation X.

Discriminative and Generative models for classification

Classification problem

- Learn a function X → Y to determine if an instance X belongs to the class Y.

(features): x

- Heigth
- Weight
- Color

(Class label): y

- Bird
- Dog

Discriminative classifier

- Learns the conditional probability P(y|x), i.e. the probability of the class label given the characteristics x.

(features): x

- Heigth
- Weight
- Color

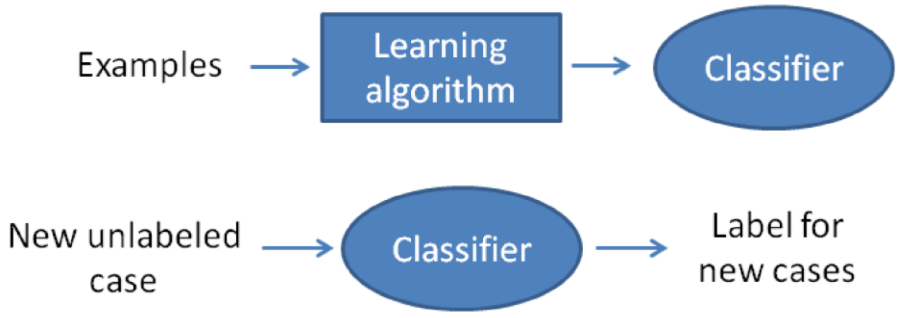
(Class label): y

- Bird
- Dog

Discriminative Model

- Discriminative classifiers estimate parameters of decision boundary/class separator directly from labeled examples.
- Tries to solve: How do I separate the classes?
- Learns $p(y|x)$ directly.
- Learn mappings from inputs to classes.

- Builds a mechanism that allows to discriminate (separate) cases of different classes without any assumption about data.



Generative classifier

- Builds a model to describe how the data looks like for a class, i.e. model the distribution of inputs characteristics for a label.
- Model the probability P(x, y). Given p(y) we can compute P(x|y).

(features): x

- Heigth
- Weight
- Color

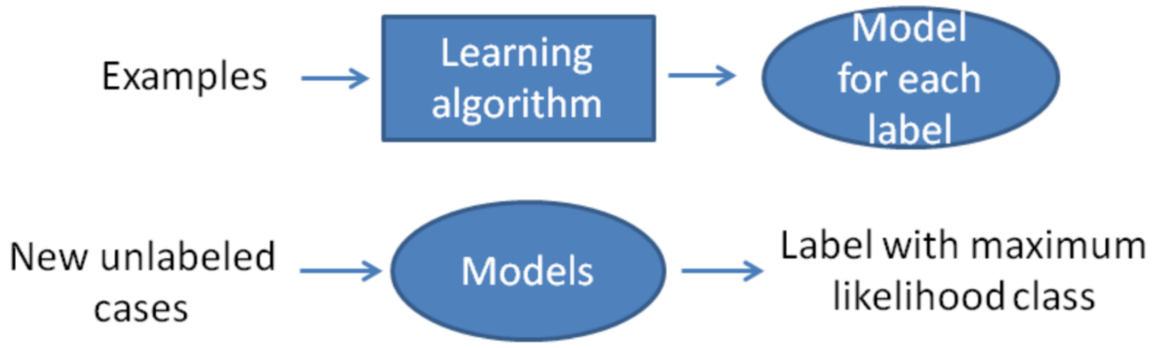
(Class label): y

- Bird
- Dog

Generative Model

- Models the distribution of inputs characteristic of the class.
- Tries to solve: What does each class ”look” like?
- Build a model of p(x|y), p(y)
- Apply Bayes Rule to calculate the posterior probability P(y=0|x) and P(y=1|x).

- Assumes a parametric distribution of data, learn the parameters for the distribution (model the classes) and, when needed, find the most likely class for new observations.

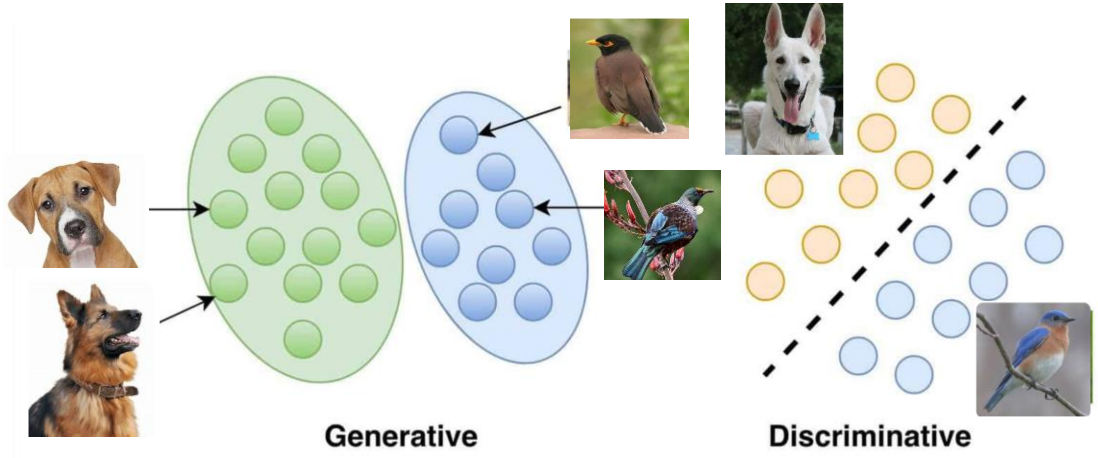


Examples

Detect the language in a text

- Generative model: Learn each language and determine to which language the text most probably belongs.
- Discriminative aproach: determine the lingüístic differences without learning any Language.

Classification of images



Comparison

Discriminative models

- Model the decision boundary for the dataset classes.
- Learn the conditional probability – $p(y|x)$.
- Computationally cheap.
- Robust to outliers, unlike generative models.

Generative models

- Aim to capture the actual distribution of classes in the dataset.
- Predict the joint probability distribution – $p(x,y)$.
- Computationally expensive.
- Impacted by the presence of outliers.

Algorithms

Discriminative models

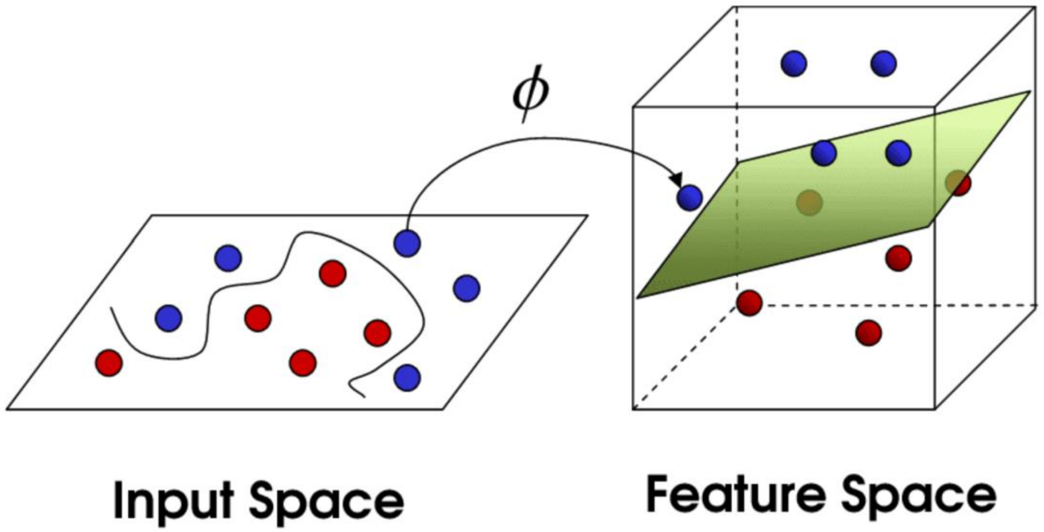
- Support Vector Machines
- Logistic Regression
- Decision Tree
- Random Forest
- Neural networks

Generative models

- Bayesian Networks
- Naive Bayes
- Linear Discriminant Analysis
- Hidden Markov Models

Support Vector Machines

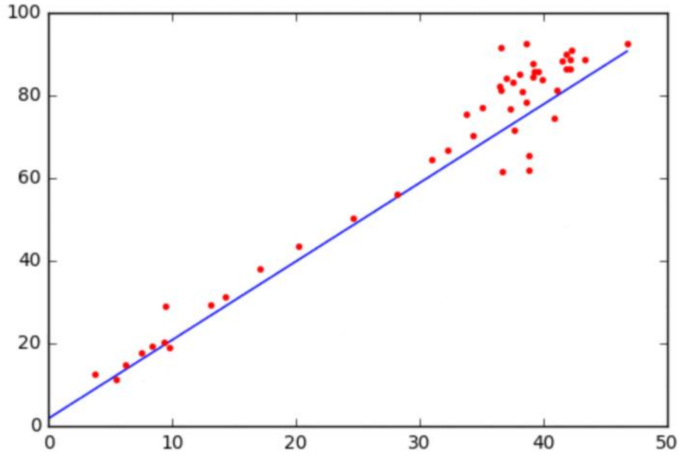
- SVM can separate both linear and non-linear data points.
- A **kernel-trick** helps separate non-linear points. Having no kernel, the **Linear SVM** finds the hyperplane with the maximum margin-linear solution to the problem.
- The boundary points in the feature space are called support vectors .Based on their relative position, the maximum margin is derived and an optimal hyperplane drawn at the midpoint.



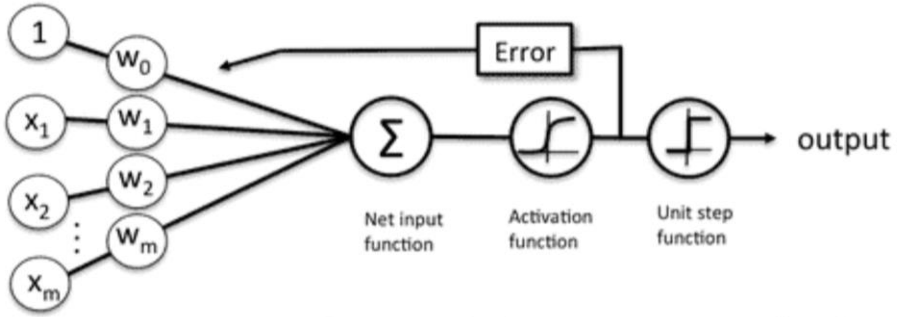
Logistic Regression

- *Linear Regression* approximates a linear decision boundary.
- *Logistic Regression* is a parametric, supervised-learning algorithm used to solve classification problems.
- Addition of a *Logistic Function* to Linear Regression built Logistic Regression.

Linear regression



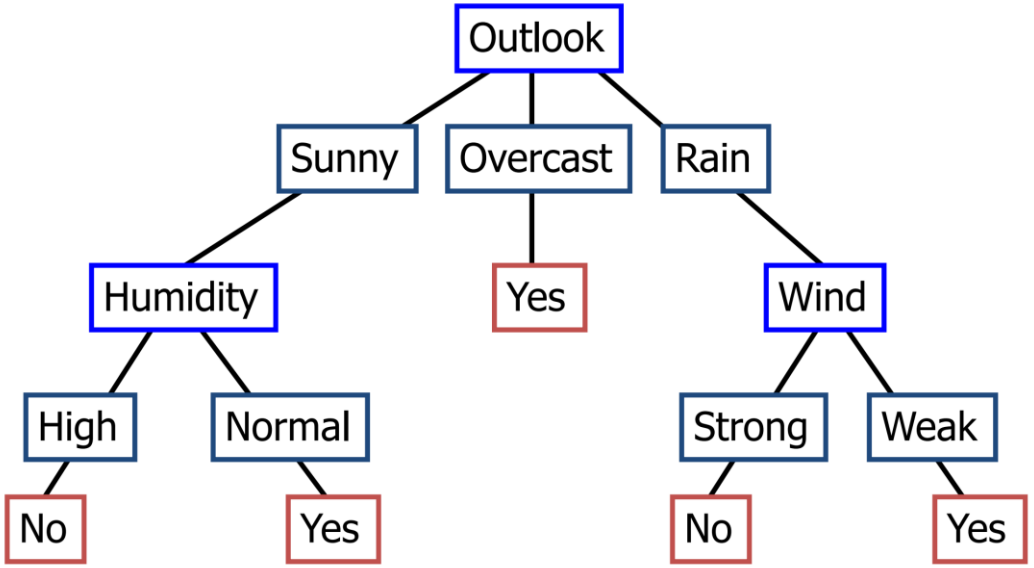
Logistic regression



Schematic of a logistic regression classifier.

Decision Tree

- Non-parametric, supervised-learning algorithm (classification or regression).
- The decision tree progressively splits the data into smaller groups, based on certain attributes, until they reach an end, where the data can be termed a label.
- Tree-structured classifier, consisting of a root node, an intermediate or decision node and a leaf node.



Random Forest

Combines multiple Decision Trees This brings the weak classifiers together and construct a more robust classifier.